LÓGICA DE PROGRAMAÇÃO APLICADA À WEB

Retomando a Aula Anterior

- Programar é resolver problemas
- Algoritmos organizam soluções
- Aplicações Web recebem dados, processam e retornam resultados
- 📌 Hoje vamos aprender a estruturar essa lógica.

Programas Executam Passos

Um programa:

- Começa no primeiro passo
- Executa instruções em sequência
- Finaliza ao terminar os passos

📌 Essa execução sequencial é a base de toda programação.

-

Estrutura Sequencial

Estrutura sequencial significa:

- O código é executado linha por linha, na ordem em que foi escrito.
- Sem decisões e sem repetições (por enquanto).

Modelo Básico de Solução

Todo algoritmo pode ser visto como:

- Entrada → Processamento → Saída

Esse modelo vale para:

- Programas simples
- Sistemas Web
- Aplicações complexas

Entrada de Dados

Entrada é qualquer dado fornecido ao sistema.

Exemplos em aplicações Web:

- Campos de formulário
- Botões
- Seleções do usuário

📌 Sem entrada, não há interação.

Processamento

Processamento é onde:

- Regras são aplicadas
- Cálculos são realizados
- Dados são analisados

📌 É aqui que a lógica acontece.

Saída de Dados

Saída é o resultado apresentado ao usuário.

Pode ser:

- Uma mensagem
- Uma página Web
- Um valor calculado

HTML

O que é HTML

- HTML (HyperText Markup Language) é uma linguagem de marcação.
- Define estrutura e significado do conteúdo (semântica).
- O navegador interpreta o HTML e renderiza a página.
- CSS define aparência; JavaScript define comportamento.

Como testar HTML

- Crie um arquivo com extensão .html (ex.: index.html).
- Abra no navegador (duplo clique) ou use “Live Server” no VS Code.
- Atualize a página para ver as mudanças.

<!DOCTYPE html>

Sintaxe

```
<!DOCTYPE html>
```

Funcionalidade

Declara o documento como HTML5 e evita “modo compatibilidade” (quirks).

Exemplo

```
<!DOCTYPE html>
<html>
  ...
</html>
```

<html lang="pt-BR">

Sintaxe

```
<html lang="pt-BR">
</html>
```

Funcionalidade

Elemento raiz do documento. O atributo lang informa o idioma (acessibilidade e SEO).

Exemplo

```
<html lang="pt-BR">
  <head>...</head>
  <body>...</body>
</html>
```

<head>

Sintaxe

```
<head>
</head>
```

Funcionalidade

Contém metadados e configurações (título, charset, viewport, links para CSS etc.).

Exemplo

```
<head>
  <meta charset="UTF-8">
  <title>Minha página</title>
</head>
```

<meta charset="UTF-8">

Sintaxe

```
<meta charset="UTF-8">
```

Funcionalidade

Define a codificação de caracteres. Garante acentos e caracteres especiais.

Exemplo

```
<head>
  <meta charset="UTF-8">
</head>
```

<meta name="viewport">

Sintaxe

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Funcionalidade

Ajusta escala/largura em dispositivos móveis e é base para responsividade.

Exemplo

```
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
```

<title>

Sintaxe

```
<title>Texto</title>
```

Funcionalidade

Define o título exibido na aba do navegador (e nos favoritos).

Exemplo

```
<head>
  <title>Site do Curso</title>
</head>
```

<body>

Sintaxe

```
<body>
</body>
```

Funcionalidade

Contém o conteúdo visível da página (textos, imagens, links, etc.).

Exemplo

```
<body>
  <h1>Olá!</h1>
  <p>Bem-vindo.</p>
</body>
```

Estrutura mínima completa

Sintaxe

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Minha primeira página</title>
</head>
<body>
  <h1>Olá, HTML!</h1>
  <p>Minha primeira página.</p>
</body>
</html>
```

```html
<!-- Salve como index.html e abra no navegador -->
<!-- Edite o conteúdo dentro do <body> -->
```

Títulos: <h1> a <h6>

Sintaxe

```
<h1>...</h1>
<h2>...</h2>
...
<h6>...</h6>
```

Funcionalidade

Define hierarquia de títulos. Ajuda estrutura, SEO e leitores de tela.

Exemplo

```
<h1>Curso Web</h1>
<h2>Aula 1</h2>
<h3>Exercícios</h3>
```

Parágrafo: <p>

Sintaxe

```
<p>Texto</p>
```

Funcionalidade

Cria parágrafos (blocos). O navegador adiciona espaçamento padrão.

Exemplo

```
<p>Aprendendo HTML no laboratório.</p>
```

Quebra de linha:

Sintaxe

```
<br>
```

Funcionalidade

Quebra de linha dentro do mesmo bloco de texto.

Exemplo

```
<p>Linha 1<br>Linha 2</p>
```

Linha horizontal: <hr>

Sintaxe

```
<hr>
```

Funcionalidade

Separador horizontal; indica mudança de seção/tema.

Exemplo

```
<h2>Sobre</h2>
<hr>
<p>Texto...</p>
```

Destaque semântico: e

Sintaxe

```
<strong>...</strong>
<em>...</em>
```

Funcionalidade

strong indica importância; em indica ênfase. O estilo padrão costuma ser negrito/itálico.

Exemplo

```
<p><strong>Atenção:</strong> prazo final hoje.</p>
<p>Isso é <em>muito</em> importante.</p>
```

Citação: <blockquote>

Sintaxe

```
<blockquote>...</blockquote>
```

Funcionalidade

Marca uma citação em bloco (texto citado, geralmente destacado).

Exemplo

```
<blockquote>
  "Simplicidade é o último grau de sofisticação."
</blockquote>
```

Link: <a href="...">

Sintaxe

```
<a href="URL">Texto</a>
```

Funcionalidade

Cria hyperlink para outra página, arquivo, ou âncora.

Exemplo

```
<a href="https://www.w3schools.com">Abrir site</a>
```

Link: target e rel

Sintaxe

```
<a href="..." target="_blank">...</a>
```

Funcionalidade

target="_blank" abre em nova aba.

Exemplo

```
<a href="https://google.com" target="_blank" rel="noopener">Google</a>
```

Âncora interna: href="#id"

Sintaxe

```
<a href="#contato">Contato</a>
...
<footer id="contato">...</footer>
```

Funcionalidade

Navega para um elemento da mesma página que tenha o id correspondente.

Exemplo

```
<a href="#final">Ir ao final</a>
<p>...</p>
<h2 id="final">Cheguei aqui</h2>
```

Imagem: <img>

Sintaxe

```
<img src="caminho" alt="descrição">
```

Funcionalidade

Insere imagem. alt descreve a imagem (acessibilidade e quando a imagem falha).

Exemplo

```
<img src="img/logo.png" alt="Logo do projeto" width="200">
```

Listas: <ul>/<ol> e <li>

Sintaxe

```
<ul>
  <li>Item</li>
</ul>

<ol>
  <li>Item</li>
```

Funcionalidade

ul cria lista não ordenada; ol cria lista ordenada; li define itens.

Exemplo

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JS</li>
</ul>
```

Tabela: <table>

Sintaxe

```
<table>
  ...
</table>
```

Funcionalidade

Organiza dados tabulares em linhas e colunas (não usar para layout).

Exemplo

```
<table border="1">
  <tr><td>A</td><td>B</td></tr>
</table>
```

Tabela: <tr>, <td>, <th>

Sintaxe

```
<tr>...</tr>
<td>...</td>
<th>...</th>
```

Funcionalidade

tr: linha. td: célula de dados. th: célula de cabeçalho (mais semântica).

Exemplo

```
<tr>
  <th>Nome</th>
  <th>Nota</th>
</tr>
<tr>
  <td>Ana</td>
  <td>9.0</td>
</tr>
```

Tabela: <thead> e <tbody>

Sintaxe

```
<table>
  <thead>...</thead>
  <tbody>...</tbody>
</table>
```

Funcionalidade

Separa cabeçalho e corpo, facilitando leitura e acessibilidade.

Exemplo

```
<table border="1">
  <thead>
    <tr><th>Nome</th><th>Curso</th></tr>
  </thead>
  <tbody>
    <tr><td>Ana</td><td>Web</td></tr>
  </tbody>
</table>
```

Semântica: visão geral

Sintaxe

```
<header>...</header>
<nav>...</nav>
<main>...</main>
<section>...</section>
<article>...</article>
<footer>...</footer>
```

Funcionalidade

Tags semânticas descrevem o papel do conteúdo (melhor organização, acessibilidade e SEO).

Exemplo

```
<main>
  <section>
    <h2>Notícias</h2>
    <article>...</article>
  </section>
</main>
```

<header> e <nav>

Sintaxe

```
<header>...</header>
<nav>...</nav>
```

Funcionalidade

header: cabeçalho da página/seção. nav: conjunto de links de navegação.

Exemplo

```
<header>
  <h1>Meu Site</h1>
  <nav>
    <a href="#">Início</a> |
    <a href="#contato">Contato</a>
  </nav>
</header>
```

<main>

Sintaxe

```
<main>...</main>
```

Funcionalidade

Representa o conteúdo principal da página (normalmente um por documento).

Exemplo

```
<main>
  <h2>Conteúdo</h2>
  <p>Texto principal...</p>
</main>
```

<footer>

Sintaxe

```
<footer>...</footer>
```

Funcionalidade

Rodapé com informações finais: contato, direitos, links úteis etc.

Exemplo

```
<footer id="contato">
  <p>Contato: contato@site.com</p>
</footer>
```

Formulário: <form>

Sintaxe

```
<form action="destino" method="GET|POST">
  ...
</form>
```

Funcionalidade

Coleta dados. GET envia pela URL; POST envia no corpo da requisição.

Exemplo

```
<form action="#" method="POST">
  <input name="nome">
  <button type="submit">Enviar</button>
</form>
```

Campo de texto: <input type="text">

Sintaxe

```
<input type="text" name="nome">
```

Funcionalidade

Campo de texto simples. name define a chave do dado enviado.

Exemplo

```
<label>Nome:</label>
<input type="text" name="nome" required minlength="3">
```

Email: <input type="email">

Sintaxe

```
<input type="email" name="email">
```

Funcionalidade

Campo com validação nativa de formato de email.

Exemplo

```
<label>Email:</label>
<input type="email" name="email" required>
```

Número: <input type="number">

Sintaxe

```
<input type="number" name="idade" min="0" max="120">
```

Funcionalidade

Campo numérico com validação por faixa (min/max).

Exemplo

```
<label>Idade:</label>
<input type="number" name="idade" min="0" max="120">
```

Texto longo: <textarea>

Sintaxe

```
<textarea name="mensagem" rows="4" cols="30"></textarea>
```

Funcionalidade

Campo para texto de múltiplas linhas. rows/cols controlam tamanho inicial.

Exemplo

```
<label>Mensagem:</label>
<textarea name="mensagem" rows="4" required></textarea>
```

Checkbox: <input type="checkbox">

Sintaxe

```
<input type="checkbox" name="termos">
```

Funcionalidade

Marcação true/false. Pode ser obrigatório com required (ex.: aceitar termos).

Exemplo

```
<label>
  <input type="checkbox" name="termos" required>
  Aceito os termos
</label>
```

Botões: <button>

Sintaxe

```
<button type="submit">Enviar</button>
<button type="reset">Limpar</button>
```

Funcionalidade

submit envia o formulário. reset limpa os campos (valores iniciais).

Exemplo

```
<button type="submit">Enviar</button>
<button type="reset">Limpar</button>
```

Validações nativas (atributos)

Sintaxe

```
required
minlength="N"
min="N" max="N"
```

Funcionalidade

Atributos de validação do HTML. O navegador bloqueia envio e mostra mensagens.

Exemplo

```
<input name="nome" required minlength="3">
<input name="idade" type="number" min="0" max="120">
```

Exemplo final (1/2) — Estrutura + Tabela

Salve como index.html e abra no navegador.

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Site do Curso</title>
</head>
<body>
  <header>
    <h1>Site do Curso</h1>
    <nav>
      <a href="#sobre">Sobre</a> |
      <a href="#contato">Contato</a>
    </nav>
  </header>

  <main>
    <section id="sobre">
      <h2>Sobre</h2>
      <p>Conteúdo principal da página.</p>
    </section>

    <section>
      <h2>Alunos</h2>
```

Exemplo final (2/2) — Formulário + Rodapé

Observe as validações nativas (required, email, checkbox).

```
  <footer id="contato">
    <h2>Contato</h2>
    <form action="#" method="POST">
      <input name="nome" placeholder="Nome" required minlength="3"><br><br>
      <input name="email" type="email" placeholder="Email" required><br><br>
      <textarea name="mensagem" rows="4" placeholder="Mensagem" required></textarea><br><br>
      <label><input type="checkbox" required> Aceito os termos</label><br><br>
      <button type="submit">Enviar</button>
      <button type="reset">Limpar</button>
    </form>
  </footer>
</body>
</html>
```

Exercícios (para fechar)

- 1) Criar perfil.html com: foto (img + alt), texto “Sobre mim” e lista de interesses.
- 2) Criar uma tabela com 5 itens (produto, preço, estoque).
- 3) Criar um formulário de cadastro com nome, email, idade e checkbox de termos.

O que é um formulário?

- Formulário = forma padrão de pegar dados do usuário.
- input recebe dados
- button envia
- form define para onde enviar (action) e como enviar (method)

Introdução ao PHP

- Conceito essencial: PHP é “server-side”
  - HTML roda no navegador.
  - PHP roda no servidor e gera HTML (ou JSON etc.) como resposta.
- Exemplo mental
  - Você acessa: http://localhost/aula/index.php
    O servidor executa o PHP e entrega resultado para o navegador.

XAMPP: seu servidor local

Por que precisamos do XAMPP?

PHP não roda dentro do navegador. Ele precisa de um servidor (Apache) para interpretar o arquivo .php e devolver HTML.

A

Apache
(servidor web)

P

PHP
(linguagem no servidor)

M

MariaDB
(banco — veremos depois)

Instalação do XAMPP (passo a passo)

Checklist de instalação

- Instale o XAMPP (Apache + PHP marcados)
- Abra o XAMPP Control Panel
- Clique em Start no Apache
- No navegador, acesse: http://localhost
- Se abrir a página do XAMPP, está OK ✅

Se o Apache não iniciar, geralmente é conflito de porta (80/443). Veja a slide “Erros comuns”.

Criando seu projeto (estrutura de pastas)

Dentro do htdocs

```
htdocs/
  └── aula2/
        ├── index.php
        ├── entrada.html
        └── saida.php
```

Acesse sempre via navegador (localhost), não abrindo o .php direto no Explorer.

Primeiro PHP: testando execução

Crie um arquivo index.php e veja se o servidor está interpretando PHP.

index.php

```php
<?php
echo "PHP funcionando no XAMPP!";
?>
```

Acesso no navegador

http://localhost/aula2/index.php

Como funciona: navegador ↔ servidor

O navegador pede uma URL. O Apache executa o PHP e devolve HTML.

Tradução rápida

O arquivo .php roda no servidor. O navegador recebe o resultado (normalmente HTML) e apenas mostra na tela.

Estrutura mínima de um arquivo PHP

```
Crie index.php:
<?php
// Comentário de uma linha
/*
 Comentário de múltiplas linhas
*/
echo "Olá, PHP!”;

<?php ... ?> inicia o código PHP.
echo imprime conteúdo para a página.
```

Variáveis e tipos principais

- Regras rápidas
- Variável começa com $
- PHP é dinamicamente tipado (o tipo pode mudar)
- Nome de variável é case-sensitive ($nome ≠ $Nome)

```
<?php
$nome = "Rafael";     // string
$idade =70;          // int
$altura = 1.82;       // float (double)
$ativo = true;        // boolean
$nada = null;         // null
```

Variáveis e tipos principais

- Tipos comuns
- string: "texto"
- int: 10
- float: 10.5
- bool: true/false
- null: ausência de valor
- array: lista/mapa
- object (mais pra frente)

Strings: concatenação e interpolação

- <?php
- $nome = "Rafael";
- $idade =70;
- // Concatenação
- echo "Nome: " . $nome . " - Idade: " . $idade . "<br>";
- // Interpolação (funciona com aspas duplas)
- echo "Nome: $nome - Idade: $idade
";

Entrada de dados: GET e POST (base para “testes”)

- Crie get.php:

```
<?php

$nome = $_GET["nome"] ?? "Visitante";

$idade = $_GET["idade"] ?? "N/A";

echo "Olá, $nome!<br>";

echo "Idade: $idade<br>";
```

- Teste no navegador:
- http://localhost/aula-php/get.php?nome=Rafael&idade=35

Teste com POST (formulário) - Crie form.php

```
<!DOCTYPE html>
<html>
<head><meta charset="utf-8"><title>Form PHP</title></head>
<body>
  <h2>Formulário</h2>
  <form method="POST" action="post.php">
    <label>Nome:</label>
    <input name="nome" required>
    <br><br>

    <label>Idade:</label>
    <input name="idade" type="number" required>
    <br><br>

    <button type="submit">Enviar</button>
  </form>
</body>
</html>
```

Crie post.php:

```
<?php
$nome  = $_POST["nome"] ?? "";
$idade = $_POST["idade"] ?? "";

echo "Recebi via POST:<br>";
echo "Nome: $nome<br>";
echo "Idade: $idade<br>";
```

Operadores Aritméticos

- + Adição
- - Subtração
- * - Multiplicação
- / - Divisão
- % - Resto da Divisão (Módulo)

Operadores de atribuição

- = Atribuição simples
- += Atribuição com adição
- -= Atribuição com subtração
- *= Atribuição com multiplicação
- /= Atribuição com divisão
- %= Atribuição com módulo
- .= Atribuição com concatenação

Operadores Lógicos

- and - e lógico
- Or - ou lógico
- Xor - ou exclusivo
- ! - Não inverso
- && - e lógico
- || - ou lógico

Operadores Relacionais

- = = - Igual
- != - Diferente de
- < - Menor que
- > - Maior que
- >= - Maior ou Igual
- <= - Menor ou Igual

Operadores de Incremento e decremento

- ++ - Incremento
- -- - Decremento

Exemplo de atribuição

- $base = 3;
- $altura = 4;
- $area = $base * $altura;

- As variáveis devem ser precedidas do símbolo ($) com exceção das constantes.

Exercício guiado — 1/2: entrada.html

Crie este arquivo em htdocs/aula2/ e acesse via localhost.

entrada.html

```html
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Entrada</title>
  </head>
  <body>
    <h2>Digite algo</h2>

    <form method="post" action="saida.php">
      <label>Mensagem:</label>
      <input type="text" name="mensagem" required>
      <button type="submit">Enviar</button>
    </form>
  </body>
</html>
```

Exercício guiado — 2/2: saida.php

O PHP vai ler o valor enviado e exibir o mesmo valor na tela.

saida.php

```php
<?php
$mensagem = $_POST["mensagem"];

echo "<h2>Saída</h2>";
echo "Você digitou: " . $mensagem;
echo "<br><br>";
echo "<a href='entrada.html'>Voltar</a>";
?>
```

Testando no navegador

Passo a passo

- Confirme o Apache ligado no XAMPP
- Abra: http://localhost/aula2/entrada.html
- Digite uma mensagem e clique em “Enviar”
- Você deve ver a página saida.php mostrando o texto digitado
- Clique em “Voltar” para preencher de novo

Mini-desafios (10–15 min)

Faça sozinho(a)

- Adicione um campo “nome” e exiba: “Olá, NOME. Você digitou: MENSAGEM”.
- Adicione um campo “cidade” e exiba: “NOME mora em CIDADE.”
- Bônus: use <br> para formatar a saída.

Tarefa (para casa)

Entregar

- Criar um formulário com 3 campos: nome, idade, mensagem
- Enviar via POST para um PHP
- Exibir na tela os 3 valores recebidos (bem formatados)
- Colocar tudo na pasta htdocs/aula2_tarefa